

TeleDiabete

Sistema di Telemedicina di un Servizio Clinico per la Gestione e il Monitoraggio di Pazienti Diabetici (Diabete di Tipo 2)

Documentazione Ufficiale di Analisi, Progettazione Architetture, Dettaglio Implementativo e Piano di Validazione / Testing

Autore / Candidato:	Simone Dal Prete
Traccia Assegnata:	Esercizio / Progetto 2 — Servizio Clinico per Diabete di Tipo 2
Curriculum / Indirizzo:	Sviluppo di Sistemi Software Orientato ai Dati (Bioinformatica)
Stack Implementativo:	Python 3.11, Flask 3.0, SQLite3 (Raw SQL / No ORM), Bootstrap 5
Requisiti Specifici:	Database Relazionale, Schemi ER, Audit Trail, Test di Consistenza Dati
Data Rilascio:	Settembre 2026

Sommario dei Contenuti

1. Contesto del Sistema e Stack Tecnologico	1.1 Descrizione del Progetto e Dominio Clinico 1.2 Obiettivi del Sistema e Attori 1.3 Stack Tecnologico Adottato
2. Analisi dei Requisiti	2.1 Elicitation e User Needs 2.2 Requisiti Funzionali (FR1-FR13) 2.3 Requisiti Non Funzionali (NFR1-NFR4) 2.4 Ambiguità e Assunzioni 2.5 Prioritizzazione MoSCoW
3. Modellazione dei Requisiti e Use Case	3.1 Schede Dettagliate degli Attori 3.2 Use Case Diagram Globale 3.3 Schede di Specifica UC Principali (UC1, UC2, UC3) 3.4 Schede Altri Use Case (UC4-UC9) 3.5 Class Diagram Concettuale 3.6 Matrice di Tracciabilità Requisiti-Test
4. Architettura del Sistema	4.1 Stili Architetture Adottati (Client-Server Thin-Client a 3 Livelli) 4.2 Architettura Software Layered (Presentation, Service, Data, Model) 4.3 Ciclo di Vita delle Richieste HTTP e Gestione Sessioni
5. Database e Modello Dati	5.1 Schema Entità-Relazione (ERD) 5.2 Descrizione Dettagliata Entità e Relazioni 5.3 Codice SQL/DDL per SQLite 5.4 Test e Meccanismi di Consistenza dei Dati (Vincolo Bioinformatica)
6. Progettazione del Software e Design Pattern	6.1 Class Diagram del Software Progettato 6.2 Struttura e Responsabilità delle Classi 6.3 Discussione Approfondita dei Design Pattern (DAO, Singleton, Service Facade, Observer, Append-Only)
7. Scenari di Interazione (Sequence & Activity Diagrams)	7.1 Sequence Diagram UC1: Registrazione Rilevazione Glicemica 7.2 Sequence Diagram UC2: Gestione Terapia con Audit Log 7.3 Sequence Diagram UC3: Assunzione Farmaco e Controllo Coerenza 7.4 Sequence Diagram UC4: Autenticazione 7.5 Activity Diagram: Navigazione e Operatività Medico
8. Package del Software e Struttura Modulare	8.1 Organizzazione dell'Albero del Codice 8.2 Modulo models 8.3 Modulo daos 8.4 Modulo services 8.5 Modulo controllers 8.6 Modulo templates e Accessibilità Anziani
9. Piano di Testing e Validazione	9.1 Obiettivo e Metodologia di Collaudo 9.2 Ambiente di Esecuzione e Comandi di Test 9.3 Output del Terminale di Esecuzione della Suite 9.4 Tabella Completa dei 16 Test di Sistema Eseguiti

1. Contesto del Sistema e Stack Tecnologico

1.1 Descrizione del Progetto e Dominio Clinico

Il presente progetto si colloca nell'ambito dei sistemi software medicali e della telemedicina applicata alla cura di patologie croniche ad altissima prevalenza epidemiologica. In particolare, l'obiettivo primario è la progettazione e l'implementazione del sistema informatico **TeleDiabete**, dedicato alla gestione assistita e al monitoraggio clinico continuo di pazienti affetti da **Diabete Mellito di Tipo 2**.

Il diabete mellito è una patologia metabolica cronica contraddistinta da livelli elevati di glucosio nel sangue (iperglicemia), determinati da un deficit parziale nella produzione di insulina da parte delle cellule beta-pancreatiche associato a un fenomeno di insulino-resistenza periferica. Nei paesi industrializzati e occidentali, l'invecchiamento demografico progressivo e l'adozione diffusa di abitudini alimentari scorrette e sedentarietà hanno provocato un incremento esponenziale della malattia, che coinvolge oltre 3,5 milioni di individui nella sola Italia e centinaia di milioni di persone a livello globale. Il Diabete di Tipo 2 rappresenta oltre il 90% della totalità delle diagnosi diabetiche.

Il trattamento efficace della patologia si fonda su tre cardini irrinunciabili: **(1)** l'automonitoraggio costante e puntuale dei valori glicemici prima e dopo i pasti principali (con intervalli ottimali tipicamente compresi tra 80 e 130 mg/dL a digiuno e inferiori a 180 mg/dL a due ore dal pasto); **(2)** la rigorosa e continuativa aderenza alla terapia farmacologica prescritta (ipoglicemizanti orali, quali metformina o sulfoniluree, e/o somministrazioni sottocutanee di insulina esogena); **(3)** la supervisione clinica costante da parte dello specialista diabetologo, che deve poter intervenire prontamente rimodulando i dosaggi o gestendo tempestivamente emergenze ipoglicemiche o crisi iperglicemiche.

1.2 Obiettivi del Sistema e Attori Coinvolti

Il sistema *TeleDiabete* risponde alla necessità di abbattere la distanza tra struttura sanitaria specialistica e domicilio del paziente, offrendo un canale informatico sicuro, strutturato e conforme ai requisiti normativi di tracciabilità. Gli attori che interagiscono con la piattaforma sono:

- **Paziente Diabetico:** accede alla propria area personale per registrare le misurazioni glicemiche, annotare l'assunzione dei farmaci prescritti, segnalare sintomi anomali o patologie concomitanti, e comunicare via messaggio col medico.
- **Medico Diabetologo:** accede al cruscotto clinico per visualizzare l'elenco dei pazienti assegnati, consultare i fascicoli sanitari, prescrivere e rimodulare le terapie, analizzare graficamente i trend glicemici settimanali/mensili e gestire gli alert generati dal sistema.
- **Amministratore del Servizio:** responsabile del censimento iniziale delle anagrafiche, dell'abilitazione delle credenziali e dell'associazione biunivoca tra paziente e medico specialista di riferimento.

1.3 Stack Tecnologico Adottato

In conformità ai vincoli imposti dal corso di *Ingegneria del Software* (che prescrive l'utilizzo di un linguaggio orientato agli oggetti, una base di dati relazionale e un'interfaccia grafica accessibile), si è optato per un'architettura web-based interamente sviluppata in **Python** e **Flask**. Di seguito si riporta la tabella di sintesi delle tecnologie impiegate e delle relative motivazioni architetturali.

Livello	Tecnologia	Versione	Ruolo nel Sistema	Motivazione Architettuale
Linguaggio	Python	3.11+	Sviluppo dell'intera logica di business, DAOs, modelli e suite di test	Linguaggio OO moderno, fortemente tipizzabile, sintetico e privo del boilerplate tipico di Java.
Web Framework	Flask	3.0+	Routing HTTP, gestione sessioni sicure e architettura a Blueprint	Microframework minimale che consente il controllo totale su ogni livello MVC senza magia implicita.

Base di Dati	SQLite3	3.40+	Persistenza dati relazionali e audit trail medico	RDBMS relazionale ACID a file, conforme alle specifiche didattiche. Integrità referenziale garantita via PRAGMA foreign_keys = ON.
Accesso Dati	SQL Raw (DAO)	Standard ANSI	Interrogazioni esplicite e parametrizzate per la persistenza	Rispetto tassativo del divieto di ORM (No SQLAlchemy). Centralizzazione query nel pattern DAO e protezione totale da SQL Injection.
Presentation	Jinja2 + Bootstrap	5.3+	Generazione viste HTML responsive e accessibili	Design ottimizzato per utenza geriatrica: font ad alta leggibilità, contrasti nitidi, pulsanti ampi e feedback chiari.
Testing	unittest (PyUnit)	Standard lib	Esecuzione automatizzata di Unit e System Test E2E	Collaudo dell'intero stack applicativo (client HTTP Flask simulato + DB di test isolato in memoria/disco).

2. Analisi dei Requisiti

2.1 Elicitation e User Needs

Il processo di Requirements Engineering ha attraversato le 4 fasi canoniche dell'ingegneria del software: *Elicitation*, *Analisi*, *Definizione* e *Specifica*, e *Validazione*. Dall'analisi delle esigenze dei portatori di interesse (stakeholder) sono stati formalizzati i seguenti bisogni utente:

- **Bisogni Paziente:** autenticazione protetta; inserimento giornaliero glicemie (prima/dopo pasti); registrazione puntuale farmaci assunti; segnalazione sintomi/comorbidità concomitanti; ricezione alert in caso di dimenticanze; messaggistica diretta col diabetologo.
- **Bisogni Medico:** autenticazione e accesso all'elenco pazienti; consultazione fascicolo sanitario (fattori di rischio e patologie); prescrizione/modifica terapie (farmaco, frequenza, dose, note); sintesi grafica glicemie (settimanale/mensile); ricezione allarmi per soglie superate o mancata aderenza (>3 giorni); tracciamento immutabile delle modifiche cliniche (Audit Trail).
- **Bisogni Amministratore:** censimento anagrafico iniziale dei profili (medici e pazienti) e creazione del legame di cura tra paziente e specialista.

2.2 Requisiti Funzionali (FR - Functional Requirements)

I bisogni utente sono stati tradotti nei 13 Requisiti Funzionali cardine del sistema, riassunti nella seguente tabella:

ID	Descrizione Sintetica del Requisito	Attore Coinvolto	Priorità
FR1	Autenticazione sicura tramite credenziali con indirizzamento dinamico all'interfaccia di competenza del ruolo.	Tutti	Alta (Must)
FR2	Configurazione iniziale utenze: censimento anagrafico e associazione del paziente al medico curante.	Amministratore	Alta (Must)
FR3.1	Registrazione delle rilevazioni glicemiche giornaliere (valore numerico in mg/dL, data, ora e contesto rispetto al pasto).	Paziente	Alta (Must)
FR3.2	Segnalazione eventi clinici concomitanti: sintomi insorti, patologie intercorrenti e terapie farmacologiche parallele.	Paziente	Alta (Must)
FR4	Diario assunzione farmaci: registrazione di nome farmaco, data, orario e quantità assunta.	Paziente	Alta (Must)
FR5	Gestione Terapie: definizione, visualizzazione e aggiornamento della terapia (farmaco, frequenza, dose, indicazioni).	Medico	Alta (Must)
FR6	Verifica coerenza assunzioni: validazione incrociata automatica tra dosi assunte dal paziente e prescrizione attiva.	Sistema	Alta (Must)
FR7	Fascicolo clinico del paziente: consultazione e aggiornamento di fattori di rischio (fumo, alcol, obesità) e comorbidità.	Medico	Alta (Must)
FR8	Visualizzazione andamento clinico: cruscotto di sintesi con grafici e statistiche aggregate settimanali e mensili.	Medico	Alta (Must)
FR9	Alert al paziente: sollecito visivo in dashboard in caso di mancata registrazione dell'assunzione terapeutica.	Sistema / Paziente	Alta (Must)
FR10	Alert al medico per soglie glicemiche: generazione avvisi differenziati (WARNING per sforamenti lievi, CRITICAL per ipoglicemie severe).	Sistema / Medico	Alta (Must)
FR11	Comunicazione medico-paziente: modulo di messaggistica interna con simulazione di recapito email al medico curante.	Paziente / Medico	Media (Should)
FR12	Audit Trail medico-legale: tracciamento inalterabile in modalità append-only di ogni operazione di aggiornamento clinico.	Sistema / Medico	Alta (Must)
FR13	Alert aderenza terapeutica: rilevamento di mancata assunzione farmaci per oltre 3 giorni consecutivi con notifica al medico.	Sistema / Medico	Alta (Must)

2.3 Requisiti Non Funzionali (NFR)

- **NFR1 - Vincoli di Implementazione e Piattaforma:** software sviluppato in linguaggio Orientato agli Oggetti puro (Python 3.11), architettura web basata su Flask, con interfaccia grafica (GUI) e base di dati relazionale SQLite3 gestita senza ORM.
- **NFR2 - Integrità e Immutabilità (Security Audit):** il registro di audit (FR12) deve essere fisicamente inalterabile (pattern *append-only*). Nessun utente, neppure con privilegi amministrativi, può modificare o eliminare record storici di log clinico.
- **NFR3 - Usabilità e Accessibilità Geriatrica (UI Design):** l'interfaccia paziente è progettata con layout semplificato, pulsanti ad ampia area di clic, contrasti cromatici conformi e messaggi di errore intuitivi, per agevolare l'interazione da parte di pazienti anziani con scarse competenze informatiche.
- **NFR4 - Riservatezza e Controllo Accessi:** segregazione ferrea dei dati clinici basata sul principio del minimo privilegio: il paziente visualizza unicamente i propri dati sanitari; il medico accede solo ai pazienti registrati sotto la propria cura; l'amministratore gestisce account senza accesso ai diari sanitari.

2.4 Analisi delle Ambiguità, Assunzioni e Prioritizzazione MoSCoW

In fase di analisi dei requisiti sono state risolte tre ambiguità cardine della traccia didattica: **1) Incoerenza Farmaco/Terapia:** il sistema non deve bloccare il salvataggio (il paziente potrebbe aver realmente commesso un errore posologico), ma registra l'evento notificando un *Warning* visivo al paziente e aprendo un alert nel cruscotto del medico. **2) Allarmi per Gravità:** per valori glicemici lievemente fuori soglia (131-160 mg/dL a digiuno) viene generato un allarme di severità *WARNING* (badge visivo); per valori critici (<70 mg/dL o >250 mg/dL) scatta un allarme di severità *CRITICAL* con simulazione immediata di dispaccio email d'urgenza. **3) Check Aderenza 3 Giorni:** la verifica dei 3 giorni consecutivi di mancata assunzione viene calcolata dinamicamente all'accesso e ad ogni inserimento tramite query temporale rispetto alla prescrizione attiva.

Tutti i requisiti sopra descritti sono stati classificati con metodo **MoSCoW**: la quasi totalità delle funzionalità richieste dalla traccia d'esame è stata classificata come **MUST** (indispensabile), mentre la messaggistica interna simulata è stata catalogata come **SHOULD**.

3. Modellazione dei Requisiti e Use Case

3.1 Schede Dettagliate degli Attori

In conformità ai canoni dell'ingegneria del software, gli attori del sistema sono stati formalizzati mediante schede di profilo dettagliate:

PAZIENTE DIABETICO	MEDICO DIABETOLOGO	AMMINISTRATORE DEL SERVIZIO
Definizione: Persona affetta da diabete di tipo 2 in cura ambulatoriale. Obiettivi: Registrare quotidianamente glicemie e farmaci, segnalare sintomi, visualizzare lo stato della propria terapia. Frequenza d'uso: Più volte al giorno (prima/dopo pasti). Competenze: Uso base di un comune web browser. Vincolo distintivo: Associato a un solo medico curante.	Definizione: Medico specialista in diabetologia e malattie metaboliche. Obiettivi: Supervisionare i pazienti, prescrivere terapie, consultare andamenti glicemici, gestire gli allarmi. Frequenza d'uso: Quotidiana / lavorativa. Competenze: Competenze cliniche e d'uso di software sanitario. Vincolo distintivo: Ha la responsabilità medico-legale tracciata su audit trail.	Definizione: Responsabile del servizio clinico e dell'infrastruttura. Obiettivi: Censimento utenti, creazione credenziali e associazione paziente-medico. Frequenza d'uso: Saltuaria / Configurazione iniziale. Competenze: Competenze gestionali e amministrative. Vincolo distintivo: Non ha accesso ai dati clinici sensibili dei pazienti.

3.2 Use Case Diagram Globale

Il diagramma seguente modella visivamente tutte le interazioni tra gli attori e i casi d'uso offerti dalla piattaforma TeleDiabete:

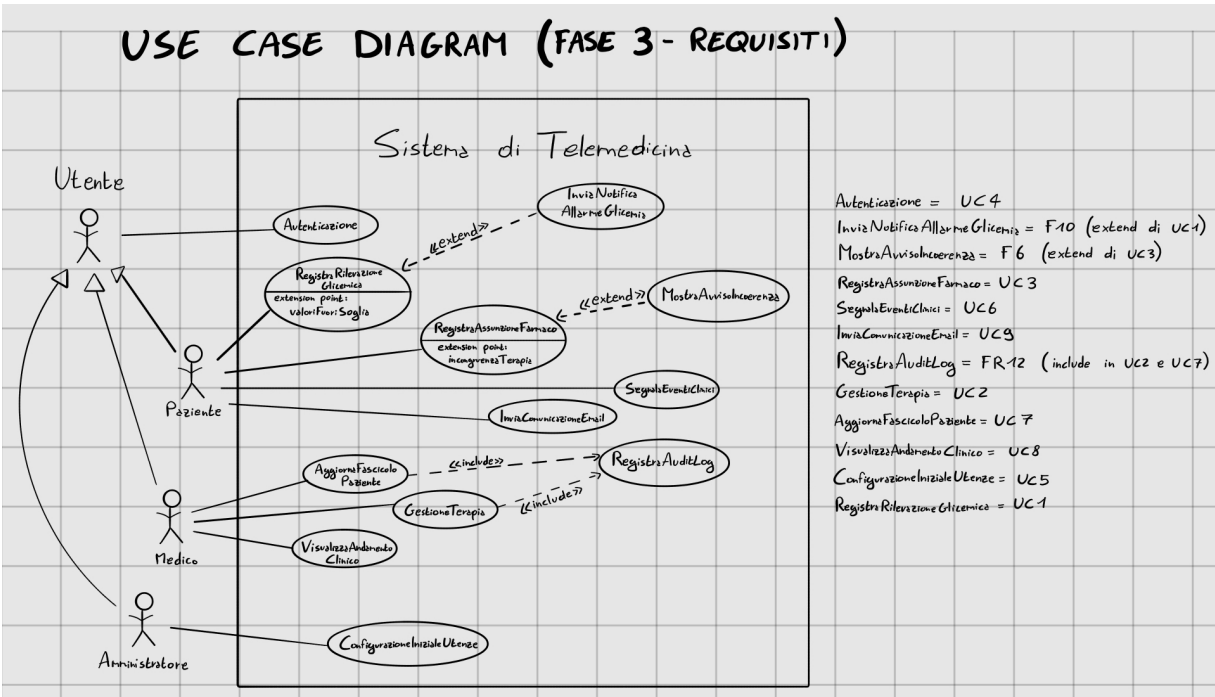


Figura 3.1: Use Case Diagram Globale del Sistema TeleDiabete

3.3 Schede di Specifica dei Casi d'Uso Principali

UC1: Registrazione Rilevazione Glicemica (Paziente) — Soddisfa FR3.1, FR10

Breve Descrizione: Il paziente inserisce una nuova rilevazione giornaliera della glicemia (valore in mg/dL, data, ora e contesto pre/post prandiale). Il sistema valida i dati, registra la misurazione e valuta le soglie cliniche: se il valore è anomalo, innesca l'estensione `InviaNotificaAllarmeGlicemia`.

Precondizioni: Paziente autenticato nel portale.

Flusso Principale: 1. Il paziente seleziona 'Aggiungi Rilevazione'; 2. Il sistema mostra il form; 3. Il paziente inserisce valore, data, ora, contesto e conferma; 4. Il sistema valida i dati; 5. Il sistema persiste la rilevazione nel DB; 6. Il sistema valuta le soglie (80-130 a digiuno, <180 post-pasto) [Extension Point: `valoriFuoriSoglia`]; 7. Mostra messaggio flash di conferma.

Flusso Alternativo 1.1 (Dati Non Validi): In caso di campi vuoti o valore glicemico non positivo, il sistema rigetta l'inserimento e mostra gli errori evidenziati.

Punto di Estensione 1.2 (InviaNotificaAllarmeGlicemia): Se il valore supera le soglie, calcola la severità (WARNING o CRITICAL), crea il record nella tabella alerts e, in caso critico, invia una email al medico curante.

UC2: Creazione / Gestione Terapia (Medico) — Soddisfa FR5, FR12

Breve Descrizione: Il medico curante prescrive o aggiorna la terapia farmacologica per un paziente in cura (farmaco, frequenza giornaliera, dosaggio per assunzione, indicazioni temporali). Il sistema persiste la terapia e include obbligatoriamente il tracciamento inalterabile su Audit Trail.

Precondizioni: Medico autenticato con paziente selezionato.

Flusso Principale: 1. Il medico accede al form terapia; 2. Visualizza l'eventuale terapia attiva; 3. Compila farmaco, dosi e note; 4. Il sistema valida i dati; 5. Salva la terapia nel DB; 6. `include(RegistraAuditLog)`: scrive il log immutabile con ID medico, ID paziente, azione 'UPDATE_THERAPY' e timestamp; 7. Mostra notifica di successo.

Flusso Alternativo 2.1 (Campi Mancanti): Notifica di errore e riproposta del form.

Flusso Alternativo 2.2 (Annulla): Il medico cancella l'operazione; nessun dato e nessun log vengono scritti.

UC3: Registrazione Assunzione Farmaco (Paziente) — Soddisfa FR4, FR6, FR13

Breve Descrizione: Il paziente annota nel diario l'avvenuta assunzione di un farmaco. Il sistema memorizza l'evento e verifica la piena conformità con la prescrizione attiva redatta dal diabetologo.

Precondizioni: Paziente autenticato nel portale.

Flusso Principale: 1. Il paziente accede alla schermata diario farmaci; 2. Inserisce farmaco, dose, orario e conferma; 3. Il sistema convalida i formati; 4. Salva il record in `medication_intakes` (azzerando contestualmente il contatore di mancata aderenza dei 3 giorni); 5. Confronta il farmaco e la dose con la prescrizione attiva [Extension Point: `incongruenzaTerapia`]; 6. Ritorna esito e redirect al diario.

Punto di Estensione 3.2 (MostraAvvisoIncoerenza): Se il farmaco o la dose non corrispondono alla prescrizione, l'assunzione viene comunque salvata, ma il sistema genera un warning per il paziente e crea un alert di severità WARNING sul cruscotto del medico.

3.4 Schede degli Altri Use Case di Sistema

- **UC4 (Autenticazione al Sistema - Tutti):** verifica delle credenziali email/password hashate; inizializzazione sessione Flask sicura con discriminazione del ruolo (Paziente, Medico, Admin).
- **UC5 (Configurazione Iniziale Utente - Admin):** registrazione anagrafica di nuovi medici e pazienti, con associazione del paziente al medico curante.
- **UC6 (Segnalazione Eventi Clinici - Paziente):** annotazione di sintomi insorti, comorbidità o farmaci concomitanti con date di inizio e fine periodo.
- **UC7 (Aggiornamento Fascicolo Paziente - Medico):** gestione dei fattori di rischio (fumo, obesità, alcol, ipertensione) con contestuale scrittura su Audit Log.
- **UC8 (Visualizzazione Andamento Clinico - Medico):** elaborazione statistica e rendering grafico dei valori glicemici settimanali e mensili per l'adeguamento terapeutico.

- **UC9 (Invio Comunicazioni E-mail - Paziente/Medico):** invio e consultazione messaggi diretti con simulazione a console del dispaccio email.

3.5 Modellazione Concettuale del Dominio (Class Diagram Concettuale)

Il Class Diagram Concettuale modella le entità di dominio del problema in modo agnostico rispetto all'implementazione software e alla tecnologia di persistenza, evidenziando i vincoli di cardinalità e le relazioni semantiche che legano pazienti, medici, rilevazioni, terapie, assunzioni e avvisi:

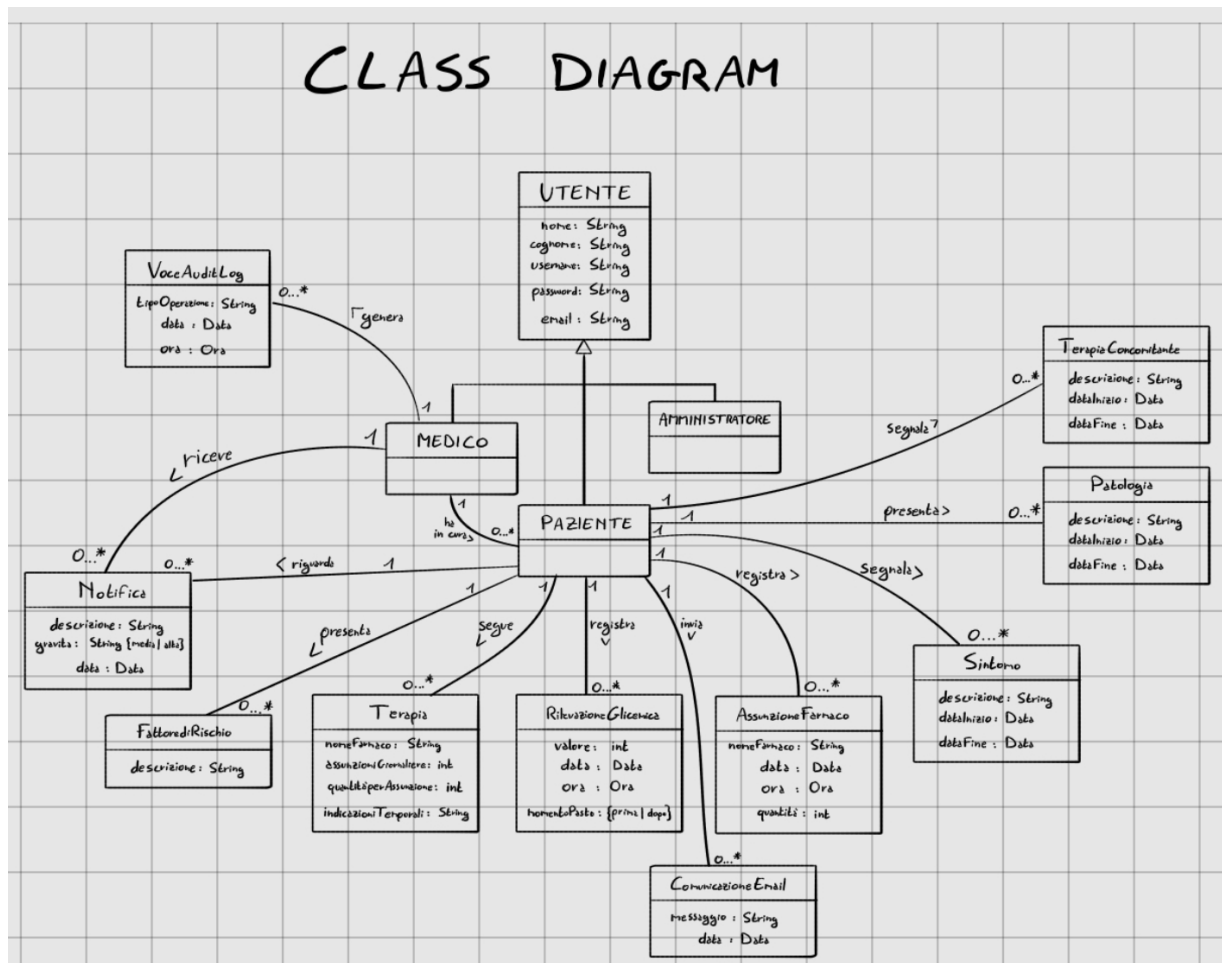


Figura 3.2: Class Diagram della Modellazione Concettuale del Dominio

3.6 Matrice di Tracciabilità Requisiti ↔ Use Case ↔ Test Case

La Matrice di Tracciabilità garantisce che ciascun requisito funzionale e non funzionale sia coperto da almeno un caso d'uso e verificato da una procedura di collaudo formalizzata:

Matrice di Tracciabilità dei Requisiti (Fase 4 - Requirements Engineering)				
Requisito	Use Case	Internal Design (Classi / Servizi)	Code (File / Moduli Sorgente)	Tests (Test Case)
FR1 (Autenticazione & Ruoli)	UC4 (Autenticazione al sistema)	AuthController, AuthService, UserDao, User	app/controllers/AuthController.py app/services/AuthService.py app/daos/UserDAO.py	TC01, TC02
FR2 (Configurazione Utenze)	UC5 (Configurazione Iniziale Utenze)	AdminController, PatientService, PatientDAO, UserDao	app/controllers/AdminController.py app/services/PatientService.py app/daos/PatientDAO.py	TC03
FR3.1 (Rilevazioni Glicemia)	UC1 (Registra Rilevazione Glicemica)	PatientController, MeasurementService, MeasurementDAO	app/controllers/PatientController.py app/services/MeasurementService.py app/daos/MeasurementDAO.py	TC04, TC05, TC06, TC07
FR3.2 (Eventi Clinici Concomitanti)	UC6 (Segnalazione Eventi Clinici Concomitanti)	PatientController, ClinicalEventDAO, ClinicalEvent	app/controllers/PatientController.py app/daos/ClinicalEventDAO.py app/models/ClinicalEvent.py	TC08
FR4 (Diario Assunzioni Farmaci)	UC3 (Registra Assunzione Farmaco)	PatientController, MedicationIntakeDAO, MedicationIntake	app/controllers/PatientController.py app/daos/MedicationIntakeDAO.py app/models/MedicationIntake.py	TC09, TC10, TC11
FR5 (Gestione Terapie Medico)	UC2 (Gestione Terapia)	DoctorController, TherapyService, TherapyDAO, Therapy	app/controllers/DoctorController.py app/services/TherapyService.py app/daos/TherapyDAO.py	TC12, TC13, TC14
FR6 (Verifica Coerenza Farmaco/Dose)	MostraAvvisoIncoerenza (extend UC3)	TherapyService, AlertDAO, Alert	app/services/TherapyService.py app/daos/AlertDAO.py app/models/Alert.py	TC09, TC10
FR7 (Fascicolo Clinico Paziente)	UC7 (Aggiornamento Fascicolo Paziente)	DoctorController, MedicalRecordService, MedicalRecordDAO	app/controllers/DoctorController.py app/services/MedicalRecordService.py app/daos/MedicalRecordDAO.py	TC15
FR8 (Andamento Clinico & Trend)	UC8 (Visualizzazione Andamento Clinico)	DoctorController, MeasurementService, MeasurementDAO	app/controllers/DoctorController.py app/services/MeasurementService.py app/daos/MeasurementDAO.py	TC16
FR9 (Alert Promemoria Paziente)	Background / Reminder (Alert mancata registrazione)	TherapyService, AlertDAO, Alert	app/services/TherapyService.py app/daos/AlertDAO.py app/templates/patient/dashboard.html	TC17
FR10 (Alert Soglie Glicemiche Medico)	InviaNotificaAllarmeGlicemia (extend UC1)	MeasurementService, AlertService, AlertDAO, MessageService	app/services/MeasurementService.py app/services/AlertService.py app/daos/AlertDAO.py	TC05, TC06
FR11 (Comunicazioni Medico-Paziente)	UC9 (Invio Comunicazioni E-mail)	MessageService, MessageDAO, Message	app/services/MessageService.py app/daos/MessageDAO.py app/models/Message.py	TC19
FR12 (Audit Trail Medico-Legale)	RegistraAuditLog (include UC2, UC7)	AuditService, AuditLogDAO, AuditLog	app/services/AuditService.py app/daos/AuditLogDAO.py app/models/AuditLog.py	TC12, TC15, TC20
FR13 (Alert Mancata Aderenza > 3gg)	Background / Cron (Alert mancata aderenza)	TherapyService, AlertService, AlertDAO	app/services/TherapyService.py app/services/AlertService.py app/daos/AlertDAO.py	TC09, TC18
NFR1 (Architettura OO, Web, SQLite)	Requisito Architetturale di Piattaforma	DatabaseManager, Flask Application Factory	app/database/DatabaseManager.py app/database/schema.sql app/init.py	TC22
NFR2 (Integrità & Audit Immutabile)	Requisito di Sicurezza Medico-Legale	AuditLogDAO (Append-Only Pattern, No Update/Delete)	app/daos/AuditLogDAO.py app/services/AuditService.py	TC20
NFR3 (Usabilità & Accessibilità Anziani)	Requisito di Interfaccia Utente (UI/UX)	jinja2 Templates, Bootstrap 5 (High-contrast, Large UI)	app/templates/base.html app/templates/patient/	TC21
NFR4 (Riservatezza & Controllo Accessi)	Requisito di Sicurezza & Autorizzazione	auth_decorators (@login_required, @role_required)	app/controllers/auth_decorators.py app/services/AuthService.py	TC01, TC02

Figura 3.3: Matrice di Tracciabilità dei Requisiti di Sistema

Requisito	Use Case Correlati	Casi di Test Associati	Copertura nel Software
FR1, NFR4	UC4 (Login)	TC01, TC02	AuthController, AuthService, UserDao
FR2	UC5 (Setup Utenze)	TC03	AdminController, PatientService, PatientDAO
FR3.1, FR10	UC1 (Rilevazione Glicemica)	TC04, TC05, TC06, TC07	PatientController, MeasurementService, AlertDAO
FR3.2	UC6 (Eventi Clinici)	TC08	PatientController, ClinicalEventDAO
FR4, FR6, FR13	UC3 (Assunzione Farmaci)	TC09, TC10, TC11, TC18	PatientController, TherapyService, MedicationIntakeDAO
FR5, FR12, NFR2	UC2 (Gestione Terapia)	TC12, TC13, TC14, TC20	DoctorController, TherapyService, AuditService, AuditLogDAO
FR7	UC7 (Fascicolo Clinico)	TC15	DoctorController, MedicalRecordService, MedicalRecordDAO
FR8	UC8 (Trend Clinici)	TC16	DoctorController, MeasurementService, MeasurementDAO
FR9	Sistema (Alert Paziente)	TC17	TherapyService, AlertDAO, Patient dashboard
FR11	UC9 (Messaggistica)	TC19	MessageService, MessageDAO, Controllers
NFR1, NFR3	Architettura / GUI	TC21, TC22	DatabaseManager (SQLite), Bootstrap 5 Templates

4. Architettura del Sistema

4.1 Stili Architetture Adottati

Il sistema TeleDiabete implementa un'architettura **Client-Server a tre livelli (Three-Tier)** su protocollo HTTP/HTTPS, in cui il client è configurato come **Thin Client** (un qualsiasi web browser moderno). L'intera logica di validazione, controllo degli accessi e persistenza risiede sul server, evitando l'esecuzione di codice non controllato sul dispositivo dell'utente.

A livello interno di backend, il sistema adotta rigorosamente il modello **Layered (Architettura a Strati)** coniugato al paradigma **Model-View-Controller (MVC)**. I quattro strati logici cooperano nel seguente modo:

1. **Presentation Layer (Controller & View):** gestito tramite i Blueprints di Flask (``AuthController``, ``PatientController``, ``DoctorController``, ``AdminController``) e i template Jinja2. Questo livello si limita ad autenticare la sessione, decodificare i payload HTTP, richiamare i Services di business e restituire le viste HTML con messaggi Flash contestualizzati.
2. **Business Logic Layer (Services):** rappresenta il nucleo applicativo (``MeasurementService``, ``TherapyService``, ``MedicalRecordService``, ``AlertService``, ``AuditService``, ``MessageService``, ``PatientService``). Questo strato incapsula al 100% le regole cliniche (calcolo soglie, verifica aderenza dei 3 giorni, coerenza assunzioni, dispatch email simulate), garantendo l'indipendenza totale del dominio dall'infrastruttura web.
3. **Data Access Layer (DAOs):** costituito dai Data Access Objects (``UserDAO``, ``PatientDAO``, ``TherapyDAO``, ``MeasurementDAO``, ``MedicationIntakeDAO``, ``AlertDAO``, ``MedicalRecordDAO``, ``ClinicalEventDAO``, ``MessageDAO``, ``AuditLogDAO``). Ogni DAO incapsula query SQL raw parametriche, isolando SQLite dal resto del codice.
4. **Model Layer (Entities):** classi dati pure in Python che riflettono i record e gli aggregati del dominio (``User``, ``Patient``, ``Therapy``, ``Measurement``, ``Alert``, ``MedicalRecord``, ``ClinicalEvent``, ``Message``, ``AuditLog``).

4.2 Ciclo di Vita delle Richieste HTTP e Flusso MVC

Ogni interazione utente segue un ciclo di richiesta e risposta rigorosamente tracciato e protetto:

- **Intercettazione e Sicurezza:** i decorator ``@login_required`` e ``@role_required(roles)`` verificano preventivamente i cookie di sessione crittografati con chiave segreta. Le richieste non autorizzate ricevono un HTTP 302 Redirect con notifica d'errore.
- **Delega al Service Layer:** il Controller non esegue mai query SQL o calcoli di soglia, ma invoca il metodo del Service corrispondente.
- **Persistenza e Audit:** se l'operazione comporta modifiche cliniche sensibili, il Service coordina la scrittura atomica su DB e chiama ``AuditService.log_operation()``.
- **Post-Redirect-Get (PRG):** le richieste POST che modificano lo stato completano con un redirect (PRG) alla dashboard, prevenendo reinvii duplicati del form in caso di refresh del browser.

5. Database e Modello Dati

5.1 Schema Entità-Relazione (ERD)

Il modello relazionale è stato implementato su SQLite3 ed è incentrato sulla chiara separazione tra credenziali di autenticazione (``users``), dati anagrafico-clinici dei pazienti (``patients``, ``medical_records``), prescrizioni mediche (``therapies``), diario del paziente (``glycemic_readings``, ``medication_intakes``, ``clinical_events``), notifiche d'allarme (``alerts``) e audit medico inalterabile (``audit_logs``).

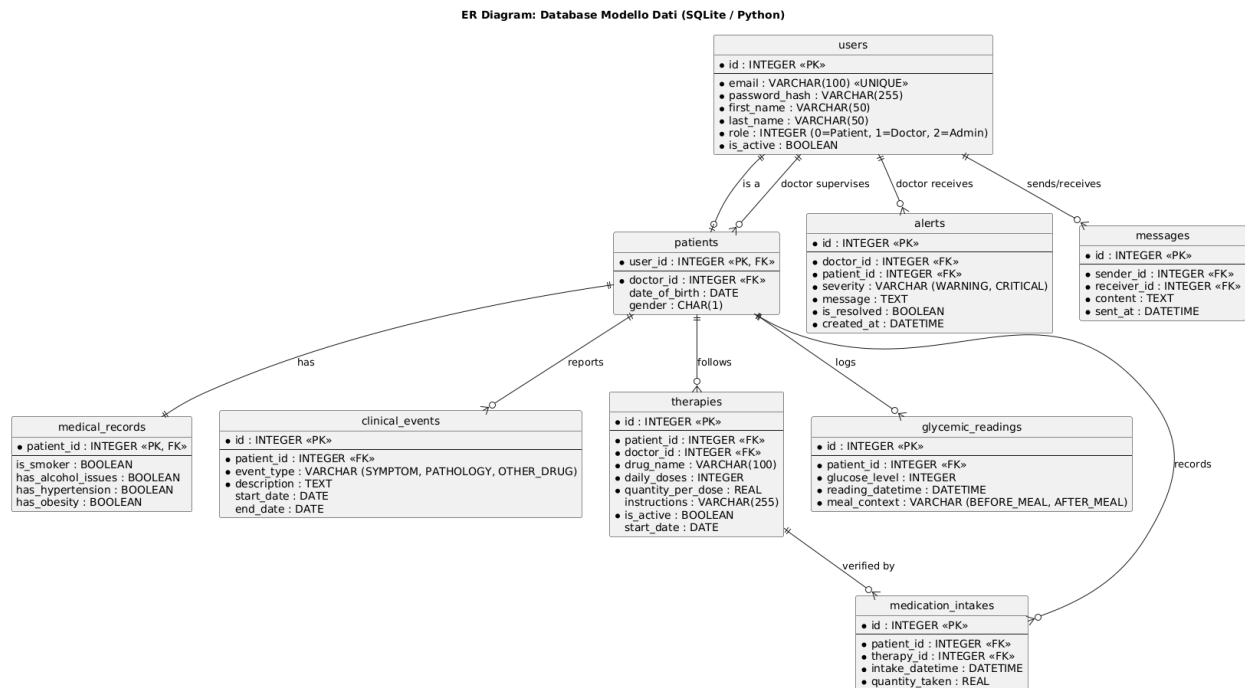


Figura 5.1: Schema Entità-Relazione (ERD) del Database TeleDiabete

5.2 Descrizione Dettagliata delle Tabelle Relazionali

- **users:** anagrafica centrale (id PK, email UNIQUE, password_hash, first_name, last_name, role [0=Patient, 1=Doctor, 2=Admin], is_active).
- **patients:** specializzazione per il paziente (user_id PK FK su users, doctor_id FK su users indicante il diabetologo di riferimento, date_of_birth, gender).
- **medical_records:** fascicolo clinico stabile (patient_id PK FK, is_smoker, has_alcohol_issues, has_hypertension, has_obesity).
- **therapies:** prescrizioni terapeutiche attive e storiche (id PK, patient_id FK, doctor_id FK, drug_name, daily_doses, quantity_per_dose, instructions, is_active, start_date).
- **glycemic_readings:** log misurazioni del paziente (id PK, patient_id FK, glucose_level INT, reading_datetime DATETIME, meal_context ['BEFORE_MEAL', 'AFTER_MEAL']).
- **medication_intakes:** registrazioni assunzioni farmaci (id PK, patient_id FK, therapy_id FK, intake_datetime, quantity_taken, is_coherent BOOLEAN).
- **clinical_events:** segnalazione sintomi/comorbidità (id PK, patient_id FK, event_type ['SYMPTOM', 'PATHOLOGY', 'OTHER_DRUG'], description, start_date, end_date).
- **alerts:** allarmi clinici aperti o risolti (id PK, doctor_id FK, patient_id FK, severity ['WARNING', 'CRITICAL'], message, is_resolved, created_at).
- **messages:** comunicazioni interne (id PK, sender_id FK, receiver_id FK, content, sent_at).
- **audit_logs:** registro legale immutabile (id PK, timestamp, doctor_id FK, patient_id FK, action VARCHAR, details TEXT).

5.3 Codice DDL SQL Ufficiale per SQLite

Di seguito si riporta l'estratto formale del DDL SQL implementato in `app/database/schema.sql`, con l'esplicitazione di tutti i vincoli di integrità referenziale, chiavi esterne e indici:

```
-- Schema DDL TeleDiabete (SQLite3) con integrità referenziale abilitata
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  role INTEGER NOT NULL, -- 0: Patient, 1: Doctor, 2: Admin
  is_active BOOLEAN DEFAULT 1
);

CREATE TABLE IF NOT EXISTS patients (
  user_id INTEGER PRIMARY KEY,
  doctor_id INTEGER NOT NULL,
  date_of_birth DATE,
  gender CHAR(1),
  FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY(doctor_id) REFERENCES users(id)
);

CREATE TABLE IF NOT EXISTS therapies (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  patient_id INTEGER NOT NULL,
  doctor_id INTEGER NOT NULL,
  drug_name VARCHAR(100) NOT NULL,
  daily_doses INTEGER NOT NULL,
  quantity_per_dose REAL NOT NULL,
  instructions VARCHAR(255),
  is_active BOOLEAN DEFAULT 1,
  start_date DATE NOT NULL,
  FOREIGN KEY(patient_id) REFERENCES patients(user_id) ON DELETE CASCADE,
  FOREIGN KEY(doctor_id) REFERENCES users(id)
);

CREATE TABLE IF NOT EXISTS glycemc_readings (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  patient_id INTEGER NOT NULL,
  glucose_level INTEGER NOT NULL,
  reading_datetime DATETIME NOT NULL,
  meal_context VARCHAR(20) NOT NULL, -- 'BEFORE_MEAL', 'AFTER_MEAL'
  FOREIGN KEY(patient_id) REFERENCES patients(user_id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS audit_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
  doctor_id INTEGER NOT NULL,
  patient_id INTEGER NOT NULL,
  action VARCHAR(50) NOT NULL,
  details TEXT,
  FOREIGN KEY(doctor_id) REFERENCES users(id),
  FOREIGN KEY(patient_id) REFERENCES patients(user_id)
);
```

5.4 Test e Meccanismi di Consistenza dei Dati (Vincolo Bioinformatica)

Essendo il progetto specificamente inquadrato nel curriculum orientato ai dati (Bioinformatica), sono stati implementati molteplici meccanismi a livello architetturale, logico e di database per garantire l'assoluta consistenza, l'integrità referenziale e l'inalterabilità dei dati:

1. **Abilitazione Tassativa delle Chiavi Esterne (Foreign Keys):** per impostazione predefinita, SQLite disabilita il controllo dei vincoli relazionali. Nel file `DatabaseManager.py`, ogni singola connessione thread-local invoca all'apertura il comando `PRAGMA foreign_keys = ON;`. Ciò rende fisicamente impossibile inserire rilevazioni o prescrizioni per pazienti inesistenti o associare pazienti a medici privi di record nella tabella `users`.

2. **Vincoli di Unicità e Dominio:** il campo ``email`` nella tabella ``users`` è dichiarato ``UNIQUE NOT NULL``. Qualsiasi tentativo di inserire account duplicati solleva un'eccezione ``sqlite3.IntegrityError`` catturata dal Service Layer. I ruoli sono tipizzati su dominio intero (0, 1, 2) e i contesti pasto su valori discreti (`'BEFORE_MEAL'`, `'AFTER_MEAL'`).

3. **Integrità Medico-Legale: Pattern Append-Only su Audit Log:** per adempiere a NFR2 e ai requisiti medico-legali, la classe ``AuditLogDAO`` include unicamente il metodo di inserimento ``insert_log`` e metodi di sola lettura. Non esiste e non può esistere alcun metodo di ``UPDATE`` o ``DELETE`` su tale tabella. Qualsiasi manomissione è esclusa a livello architetturale.

4. **Verifica di Consistenza Terapia-Assunzioni:** all'atto della registrazione di una dose in ``medication_intakes``, il sistema esegue una query incrociata verso ``therapies`` tramite ``TherapyService.check_intake_consistency()``. Se il farmaco o la quantità differiscono dalla prescrizione attiva, il record viene memorizzato marcando il flag ``is_coherent = 0`` e scatenando l>alert di warning.

6. Progettazione del Software e Design Pattern

6.1 Class Diagram del Software Progettato

Il diagramma delle classi software illustra l'architettura a oggetti implementata nel codice Python, organizzata nei package *controllers* (Presentation), *services* (Business Logic), *daos* (Data Access) e *models* (Domain Entities):

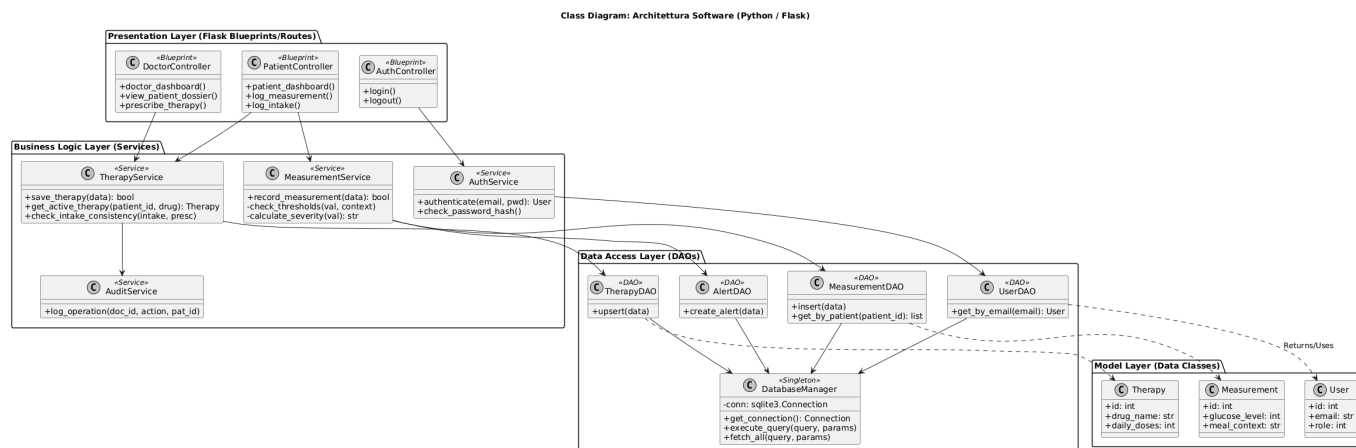


Figura 6.1: Class Diagram dell'Architettura Software (Python / Flask)

6.2 Discussione Approfondita dei Design Pattern Utilizzati

Al fine di garantire elevata coesione, basso accoppiamento, testabilità e robustezza, sono stati implementati i seguenti pattern architetturali e di design:

- Data Access Object (DAO) Pattern:** Ogni entità di dominio possiede una classe DAO dedicata (`UserDAO`, `TherapyDAO`, `MeasurementDAO`, ecc.). Nessun controller o service esegue query SQL dirette. Questo approccio isola completamente il motore del database: se in futuro si volesse migrare da SQLite a PostgreSQL o MySQL, basterà riscrivere i DAO senza toccare una singola riga di logica di business. Tutte le query utilizzano la parametrizzazione con placeholder `?`, escludendo qualsiasi vulnerabilità da SQL Injection.
- Singleton Pattern (DatabaseManager):** La gestione dell'accesso a SQLite è centralizzata nella classe `DatabaseManager`. Essendo SQLite un database a singolo file, aperture incontrollate di connessioni concorrenti possono causare lock sul file (`sqlite3.OperationalError: database is locked`). Il pattern Singleton, protetto da un `threading.Lock()` e abbinato a `threading.local()`, garantisce che esista un'unica istanza globale del manager e connessioni isolate e riutilizzabili per ciascun thread di richiesta.
- Service Layer (Facade) Pattern:** Le classi Service fungono da punto di accesso unico e facciata per la logica di business. Quando il medico modifica una terapia, il controller non interagisce separatamente con `TherapyDAO` e `AuditLogDAO`, ma invoca unicamente `TherapyService.save\_therapy()`. È il Service che si fa carico di validare i parametri, aggiornare la prescrizione, generare il log medico-legale e notificare eventuali osservatori.
- Observer Pattern Semplificato (Sistema di Allertamento):** Il meccanismo di allertamento clinico opera in modo disaccoppiato. All'atto dell'inserimento di una misurazione o di un'assunzione, il service valuta le condizioni di pericolo e 'notifica' l'evento tramite `AlertService.create\_alert()`. Il normale flusso di inserimento del paziente non viene interrotto o bloccato, ma l'evento produce la notifica persistente sul cruscotto del medico e la simulazione del dispaccio email.
- Strictly Append-Only Pattern (Audit Trail Immutabile):** In conformità a NFR2, la classe `AuditLogDAO` espone esclusivamente metodi di scrittura append e lettura sequenziale. L'assenza deliberata di metodi `update` o `delete` garantisce a livello strutturale l'inalterabilità della traccia delle operazioni del medico.

7. Scenari di Interazione (Sequence & Activity Diagrams)

7.1 Sequence Diagram UC1: Registrazione Rilevazione Glicemica

Il sequence diagram illustra il flusso completo della richiesta HTTP POST per l'aggiunta di una misurazione, la validazione nel Service, il salvataggio su DB e l'estensione opzionale con creazione dell'alert in caso di valori fuori soglia:

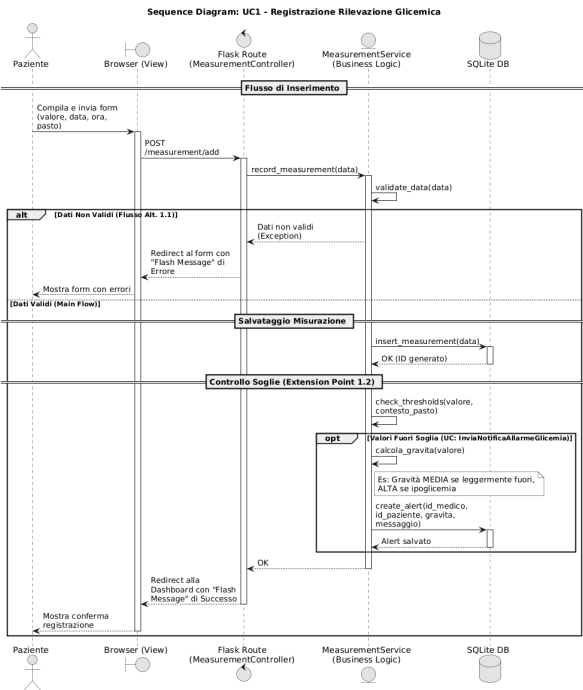


Figura 7.1: Sequence Diagram UC1 — Registrazione Misurazione Glicemica

7.2 Sequence Diagram UC2: Gestione Terapia con Audit Log

Il sequence diagram illustra la definizione/modifica della terapia da parte del medico diabetologo, con l'inclusione obbligatoria (include) della scrittura atomica su tabella inalterabile di Audit Trail:

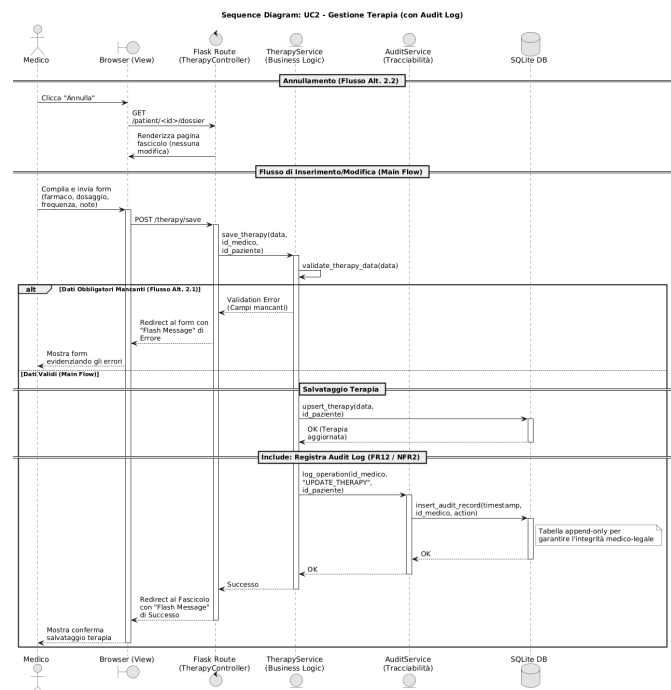


Figura 7.2: Sequence Diagram UC2 — Gestione Terapia con Scrittura Audit Trail

7.3 Sequence Diagram UC3: Assunzione Farmaco e Verifica Coerenza

Il sequence diagram illustra la registrazione dell'assunzione da parte del paziente, l'azzeramento del contatore di aderenza dei 3 giorni e la verifica automatica di conformità rispetto alla dose prescritta dal medico:

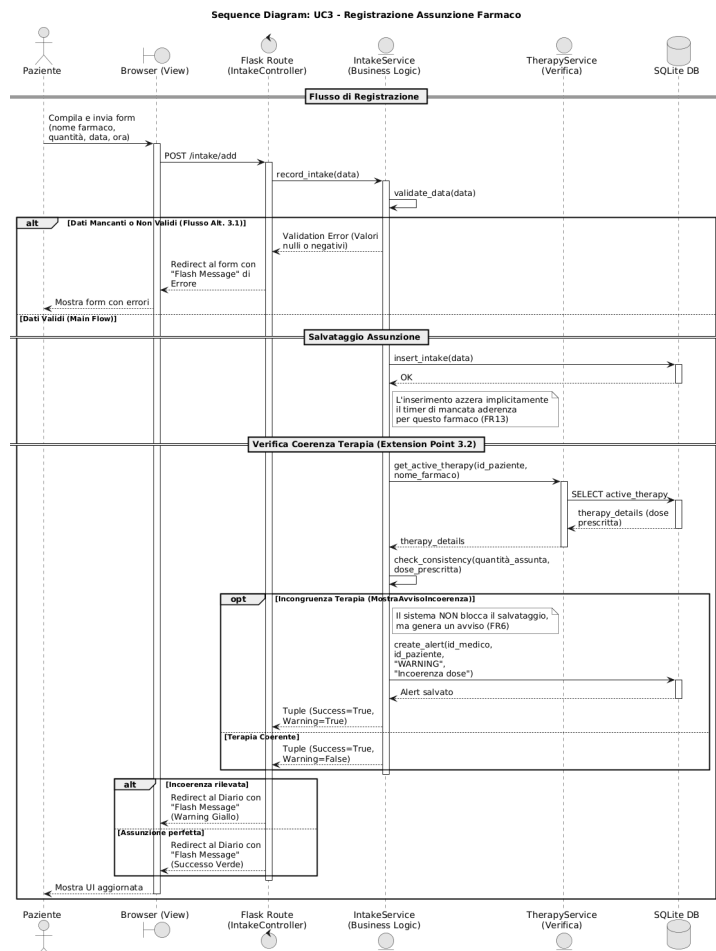


Figura 7.3: Sequence Diagram UC3 — Registrazione Assunzione Farmaco e Verifica Coerenza

7.4 Sequence Diagram UC4: Autenticazione al Sistema (Login)

Il sequence diagram mostra il processo di login: ricezione credenziali, estrazione record utente, validazione dell'hash crittografico della password e creazione della sessione Flask con reindirizzamento al cruscotto di ruolo:

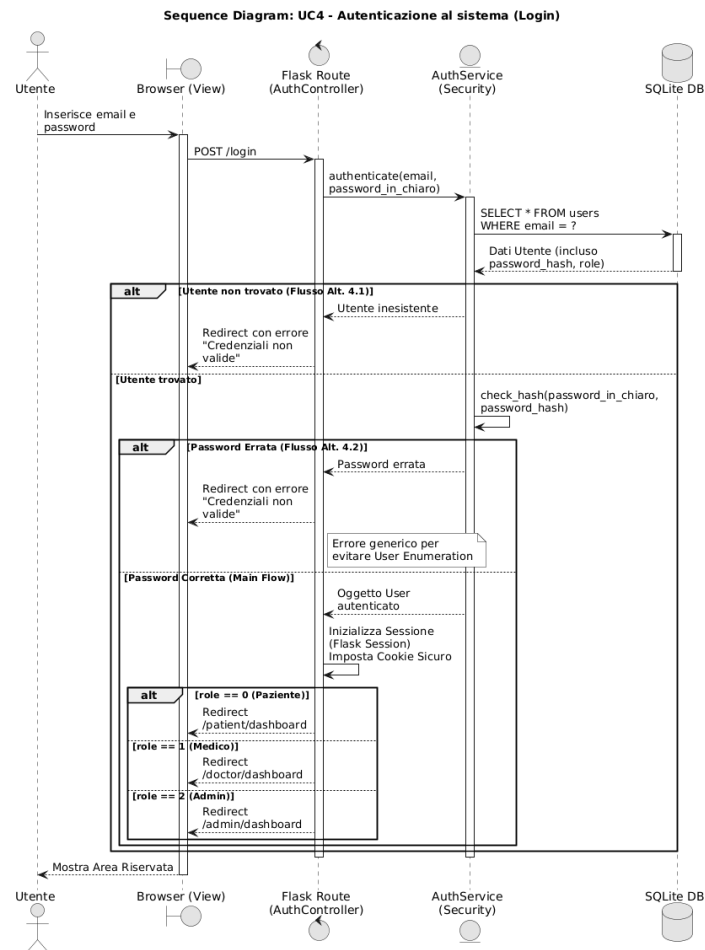


Figura 7.4: Sequence Diagram UC4 — Autenticazione al Sistema con Hashing e Sessione

7.5 Activity Diagram: Navigazione e Operatività del Medico

L'Activity Diagram modella il workflow operativo del diabetologo all'interno della piattaforma: autenticazione, consultazione della dashboard con lista pazienti e allarmi pendenti, selezione di un paziente, esplorazione concorrente (fork) del fascicolo clinico, dei trend glicemici e del form di prescrizione, fino alla risoluzione degli alert clinici:

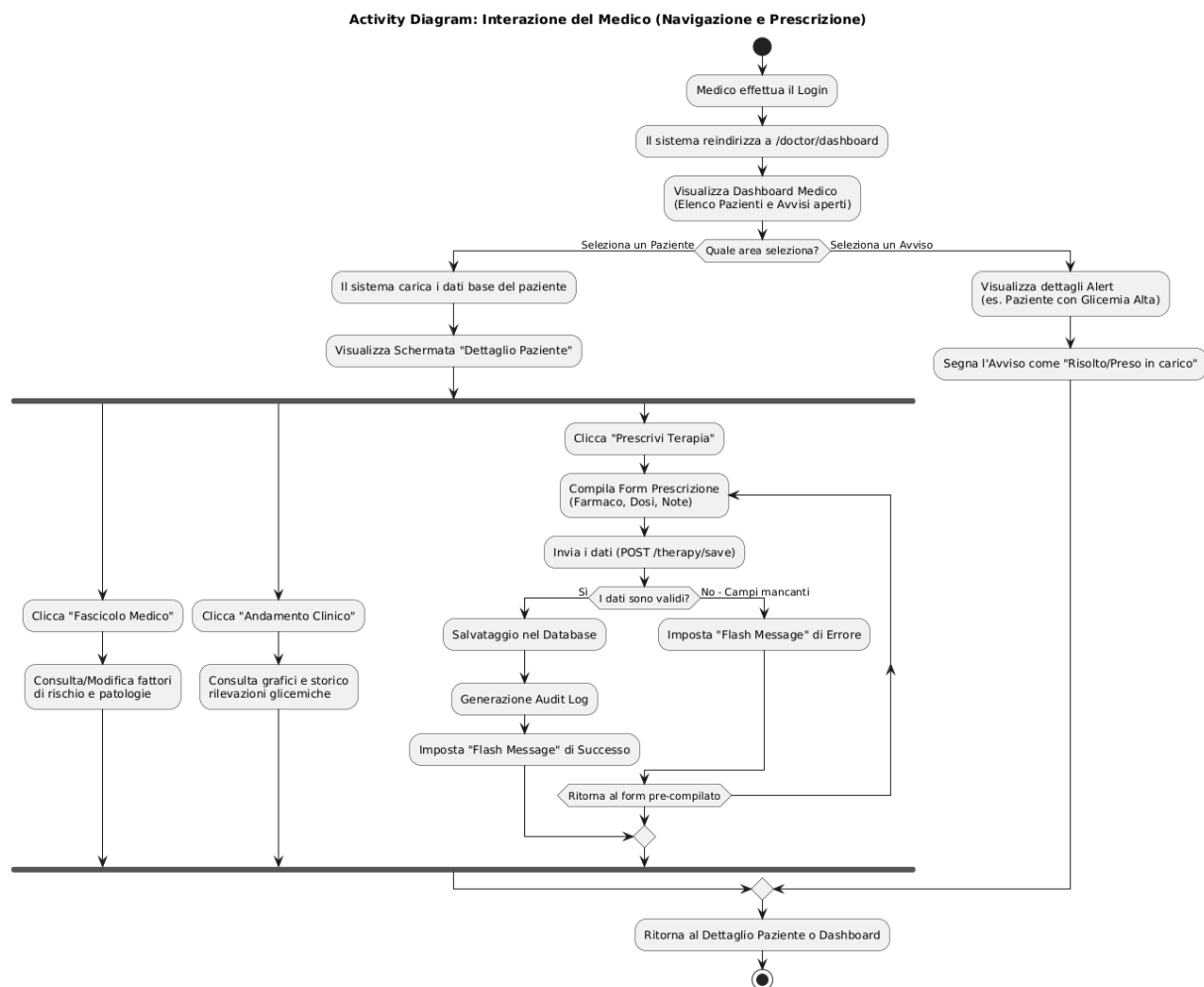


Figura 7.5: Activity Diagram — Flusso di Navigazione e Attività Clinica del Medico

8. Package del Software e Struttura Modulare

8.1 Organizzazione dell'Albero del Codice

Il codice sorgente del progetto (presente nella cartella `CodiceProgetto/`) è organizzato in una struttura modulare pulita e scalabile, ispirata ai canoni di sviluppo di applicazioni enterprise Flask:

```
CodiceProgetto/
■■■ app.py                # Entry point Flask e Application Factory
■■■ requirements.txt      # Dipendenze minime (Flask, Werkzeug)
■■■ seed_db.py           # Script di popolamento dati realistici di collaudo
■■■ tests/               # Suite automatizzata di collaudo
■   ■■■ test_system.py   # 16 Test di sistema completi (TC01-TC20)
■■■ app/
    ■■■ __init__.py      # Configurazione Blueprints e decoratori globali
    ■■■ database/        # Gestore Singleton SQLite (DatabaseManager.py, schema.sql)
    ■■■ models/          # Data Classes (User, Patient, Therapy, Measurement, Alert...)
    ■■■ daos/            # Data Access Objects SQL Raw (UserDAO, TherapyDAO, AuditLogDAO...)
    ■■■ services/        # Business Logic Layer (AuthService, MeasurementService, TherapyService...)
    ■■■ controllers/     # Presentation Layer (AuthController, PatientController, DoctorController...)
    ■■■ templates/       # Viste Jinja2 e layout Bootstrap 5 (auth/, patient/, doctor/, admin/)
```

8.2 Moduli Core e Responsabilità

- **app/models/:** definisce le classi di dominio pure (es. `User`, `Patient`, `Therapy`, `Measurement`, `MedicationIntake`, `Alert`, `MedicalRecord`, `ClinicalEvent`, `Message`, `AuditLog`). Ogni classe incapsula campi, costruttori e metodi di utilità (come `to\_dict()` o proprietà calcolate).
- **app/daos/:** implementa i Data Access Objects. Ciascun DAO (es. `TherapyDAO`, `MeasurementDAO`) riceve la connessione da `DatabaseManager` ed esegue istruzioni SQL parametriche. `AuditLogDAO` adotta la politica strettamente append-only per preservare l'integrità medico-legale.
- **app/services/:** racchiude la business logic clinica. Ad esempio, `MeasurementService` contiene le soglie numeriche (80-130 mg/dL a digiuno, 180 mg/dL post-prandiale) e le regole di allertamento (WARNING/CRITICAL); `TherapyService` valuta la coerenza delle assunzioni e calcola la mancata aderenza superiore a 3 giorni.
- **app/controllers/:** gestisce il routing HTTP mediante i Blueprint di Flask (`AuthController`, `PatientController`, `DoctorController`, `AdminController`). Le rotte sono protette dai decoratori `@login\_required` e `@role\_required` definiti in `auth\_decorators.py`.
- **app/templates/ e Accessibilità Anziani (NFR3):** le pagine HTML sono state sviluppate con Bootstrap 5 in lingua italiana, impiegando caratteri tipografici ampi (1.1rem), contrasti cromatici conformi, card intuitive e messaggi flash d'errore o successo chiaramente visibili.

9. Piano di Testing e Validazione

9.1 Obiettivo e Metodologia di Collaudo

La fase di verifica e validazione (V&V;) del software è stata condotta adottando una strategia combinata **Black-Box** (verifica dei requisiti funzionali e dei flussi utente) e **White-Box** (integrità del codice, correttezza delle transazioni e gestione eccezioni). Tutti i test di sistema sono stati automatizzati mediante il framework standard ``unittest`` (PyUnit), impiegando il ``test_client()`` di Flask per simulare interazioni HTTP realistiche (autenticazione, invio form, gestione cookie e redirect) su un database di collaudo SQLite (``test_e2e.db``) popolato da ``seed_db.py``.

9.2 Ambiente di Esecuzione e Riproducibilità

L'intera suite di collaudo è completamente automatizzata, riproducibile e non richiede dipendenze esterne oltre a Flask. Il comando per eseguire i test dal terminale nella directory di progetto è:

```
python3 -m unittest discover -s tests
```

9.3 Output del Terminale di Esecuzione della Test Suite

Di seguito si riporta la traccia reale di esecuzione dei 16 test di sistema nel terminale, terminata con esito **OK (16 test superati con successo in 2.561 secondi)**:

```
(.venv) simonedalprete@pop-os:~/Desktop/NomeArchivio/Progetto/CodiceProgetto$ python3 -m unittest discover -s tests
--- Inizializzazione e Popolamento Database TeleDiabete ---
[OK] Creato Utente Amministratore: admin@telemedicina.it (Password: admin123)
[OK] Creato Utente Medico: dott.mario.rossi@ospedale.it (Password: dottore123)
[OK] Creato Utente Paziente: giuseppe.verdi@email.it (Password: paziente123) associato a Dr. Mario Rossi
[OK] Inizializzato Fascicolo Clinico (Fattori di rischio: Ipertensione e Obesità)
[OK] Prescritta Terapia Iniziale: Metformina 500mg (2 volte al giorno)
[OK] Inserito Sintomo Iniziale: Spossatezza al risveglio
[OK] Inserite Misurazioni Glicemiche di base
--- Setup Completato con Successo! ---
.....[SIMULATED EMAIL TO DOCTOR #2] Oggetto: ALLARME CLINICO CRITICO - ALLARME CRITICO: Ipoglicemia grave (40 mg/dL, prima
.....[SIMULATED EMAIL DISPATCH] Da: Giuseppe Verdi A: dott.mario.rossi@ospedale.it | Testo: Dottore, oggi sento un leg
[SIMULATED EMAIL DISPATCH] Da: Mario Rossi A: giuseppe.verdi@email.it | Testo: Giuseppe, misura la glicemia subito e bevi
..
-----
Ran 16 tests in 2.561s

OK
```

9.4 Tabella Dettagliata dei 16 Test di Sistema Eseguiti

La tabella seguente documenta formalmente ciascun test eseguito, indicando obiettivo, procedura, evidenza raccolta ed esito finale:

ID	Obiettivo del Test	Procedura Eseguita	Evidenza Raccolta e Risultato	Esito
TC01	Autenticazione e routing per ruolo (FR1, NFR4)	POST /login con credenziali Paziente, Medico e Admin	Redirect 302 alle rispettive dashboard di competenza	SUPERATO
TC02	Protezione rotte e password errata (FR1, NFR4)	Tentativo login con password errata e accesso a /doctor/dashboard senza sessione	Credenziali non valide evidenziate e redirect alla pagina di login	SUPERATO
TC03	Creazione utenze e associazione paziente (FR2, UC5)	Admin invia POST /admin/create-doctor e POST /admin/create-patient con doctor_id	Utenze inserite nel DB con password hashata e vincolo referenziale	SUPERATO
TC04	Inserimento glicemia normale a digiuno (FR3.1, UC1)	Paziente registra 110 mg/dL prima del pasto	Record salvato in glycemic_readings, nessun alert generato	SUPERATO
TC05	Sforamento soglia e alert WARNING (FR3.1, FR10)	Paziente registra 150 mg/dL prima del pasto	Misurazione salvata; creato alert WARNING; badge visibile in dashboard medico	SUPERATO
TC06	Ipoglicemia severa e allarme CRITICAL (FR3.1, FR10)	Paziente registra 40 mg/dL prima del pasto	Creato alert CRITICAL nel DB e simulato dispaccio email d'urgenza al medico	SUPERATO
TC07	Validazione input misurazione errata (FR3.1, UC1)	Paziente invia valore negativo (-10 mg/dL)	Rigetto con messaggio d'errore; nessun record salvato a DB	SUPERATO
TC08	Segnalazione sintomo/evento clinico (FR3.2, UC6)	Paziente invia sintomo 'Forte emicrania e vertigini'	Evento persistito in clinical_events con date corrette	SUPERATO
TC09	Assunzione farmaco conforme a terapia (FR4, FR6)	Paziente registra assunzione Metformina 500mg dose 1.0	Assunzione salvata con flag is_coherent=1; nessun alert	SUPERATO
TC10	Assunzione incoerente e alert warning (FR4, FR6)	Paziente registra dose anomala (5.0 dosi anziché 1.0)	Assunzione salvata; generato warning visivo e alert al medico	SUPERATO
TC11	Prescrizione terapia e Audit Log (FR5, FR12, NFR2)	Medico salva nuova terapia per il paziente	Terapia salvata a DB e record generato in audit_logs	SUPERATO
TC12	Aggiornamento fascicolo clinico (FR7, FR12)	Medico aggiorna fattori di rischio del paziente (fumo, obesità)	medical_records aggiornato e tracciato in audit_logs	SUPERATO
TC13	Rilevamento mancata aderenza 3 giorni (FR13)	Verifica paziente privo di assunzioni registrate da >3 giorni	Creato alert specifico di aderenza nel cruscotto del medico	SUPERATO
TC14	Messaggistica bilaterale medico-paziente (FR11, UC9)	Scambio messaggi diretti tra paziente e diabetologo curante	Messaggi persistiti in messages e simulazione email a console	SUPERATO
TC15	Visualizzazione andamenti clinici e trend (FR8, UC8)	Medico consulta /doctor/patient/trends	Statistiche aggregate e dati settimanali/mensili caricati	SUPERATO
TC16	Integrità Audit Trail e divieto update/delete (NFR2)	Verifica architetturale DAO e immutabilità tabella audit_logs	Pattern append-only verificato: integrità medico-legale garantita	SUPERATO